

第5章 类和对象

本章首先介绍面向对象程序设计的基本思想及基本特征,然后介绍类的定义方法和对象的定义方法及使用。通过本章的学习,应理解面向对象程序设计的基本思想及基本特征,重点掌握类和对象的定义方法、对象成员的引用方法,熟练掌握构造函数和析构函数,掌握友员函数,掌握类模板的说明及其实例化;了解拷贝构造函数和 `this` 指针,了解静态成员和友元类,并且能够用类和对象编写解决简单问题的程序。

5.1 面向对象程序设计概述

在前面几章中,已经学习了使用 C++ 编写比较简单程序的方法,了解了基本的结构化程序设计方法。虽然这种结构化程序设计方法有很多优点,但是,作为一种面向过程的设计方法,由于它的操作与数据相分离,所以不能很好地支持原有软件的再用(软件重用),一旦数据的结构或格式发生变化、程序需要扩充或升级,都必须修改相关的操作模块。随着软件规模的急剧增大(比如达到几百万行甚至几千万行),如果每个程序都要这样修改,那么,不仅软件的生产率远远不能满足实际需要,而且大大限制了软件产业的发展。因此,面向对象程序设计方法应运而生。

5.1.1 面向对象的基本概念

面向对象程序设计方法是对结构化程序设计方法的继承和发展。下面首先介绍面向对象的基本概念。

1. 对象

对象有两方面的含义,即现实世界中的含义和面向对象程序设计中的含义。

在现实世界中,任何事物都是对象。它可以是一个有形的、具体存在的事物。例如,一台电视机、一个电视遥控器、一辆汽车、一个教师、一个学校等等。它也可以是一个无形的、抽象的事物。例如,某个日期、某个时刻、一个方案、一个计划等等。对象既可以相当简单,也可以非常复杂,复杂的对象往往由若干个简单的对象组成。整个世界都可以看成是一个异常复杂的对象。现实世界中的对象具有以下特性:

- ①每个对象必须有一个名字,以便与其他对象相区别;
- ②每个对象都有自己的某些属性(特征、状态);
- ③每个对象都有自己的一组操作(方法、功能、服务),每个操作决定对象的某种行为;
- ④对象的操作可以分成自身所承受的操作和施加在其他对象上的操作。

例如,一台电视机有大小、颜色、按钮、预存台数、价格等属性,有打开、设置、选台、跳台、调整音量等行为。而人可以通过电视遥控器与电视机通信并操纵电视,电视机根据所接收的操纵信号执行某个或某些行为。

所以,在现实世界中可以认为:对象=属性+行为。

在面向对象程序设计中,对象是封装了描述其属性的数据以及对这些数据所施加的操作而构成的统一体。因此,可以认为:对象=数据+操作。

不管是现实世界中的对象,还是面向对象程序设计中的对象,都具有数据和实现操作的隐蔽性。就像人可以通过电视遥控器操纵电视机一样,不必了解电视遥控器和电视机的内部构造,也就是说,电视遥控器和电视机的内部情况对用户来说是隐蔽的。在面向对象程序设计中,使用对象时,只需知道它所提供的外部操作接口形式,而无需知道它的操作的内部实现代码。这样,不仅使得对象的使用变得更加简单、方便,而且具有更高的安全性和可靠性。显然,面向对象程序设计中的对象来源于现实世界,更接近人们的自然思维方式。

2. 类

把具有共同属性和操作的对象进行抽象描述,就形成了面向对象程序设计的核心——类。例如,电视机类、汽车类、教师类等。在 C++ 中,用类的数据成员描述对象的特征,用类的成员函数描述对象的操作。

类是面向对象程序设计中最重要概念。类是建立对象的“模板”,对象是类的具体化,是某个类的一个实例。类是抽象的,不占用计算机的存储空间,而对象是具体的,每个对象占用独立的计算机的存储空间。就好像现实世界中的“人类”,在地球上不存在抽象的“人”,只有具体的人,如孔子、马克思、鲁迅等。

显然,必须首先说明类,描述它的数据成员和成员函数,然后才能定义、使用该类的对象。

3. 消息

对象不是孤立存在的实体,对象与对象之间存在着各种各样的联系,各种不同的联系使它们构成了各种不同的系统。对象与对象之间的联系使得它们必须相互通信,即传递消息,以便实现一定的任务。消息具有以下性质:

- ① 同一个对象可以接收不同形式的多个消息,作出不同的响应;
- ② 相同形式的消息可以传递给不同的对象,所作出的响应可以是不同的;
- ③ 对消息的响应并不是必需的,对象可以响应消息,也可以不响应。

4. 方法

方法就是对象在接收消息后所执行的操作。方法包括接口和方法体两部分。方法的接口给出了它的调用协议,方法的方法体由实现某种操作的一系列计算步骤组成。在 C++ 中,方法就是类的成员函数。方法的接口就是成员函数的函数原型,方法体就是成员函数的函数体。

5.1.2 面向对象程序设计方法的基本特征

面向对象是一种崭新的方法论。用这种方法观察世界,将客观世界看成由许多不同种类的对象组成,每类对象有特定的特征和操作,不同对象的相互作用构成了完整的客观世界。面向对象的程序设计方法就是用面向对象的观点来观察世界、用面向对象的方法来描述世界、用面向对象的计算机程序来处理现实世界中的问题。通过类和对象实现对客观世界高度的概括、分类和抽象。正因为引入类和对象,才使得面向对象的程序设计具有抽象性、封装性、继承性和多态性。

1. 抽象性

抽象是人类认识世界的一种基本方法。抽象是通过对具体的实例(对象)进行分析、归纳,抽取共性而形成概念的过程。在抽象过程中,忽略与当前研究目标无关的部分,而强调与当前研究目标有关的部分。在面向对象程序设计方法中,通过类来描述和实现对于一个具体问题的抽象分析结果。

例如,在管理学生成绩时,在属性方面,只关注学生的学号、姓名、性别以及课程名称和分数,而忽略身高、体重、爱好等;在操作方面,只关注学生成绩数据的录入、修改、查找、排序、统计、输出等,而忽略对学生其他数据的处理。如果要编写一个管理学生健康方面的程序时,关心的属性和操作就会有很大不同了。

2. 封装性

封装是指将数据和与这些数据有关的操作结合在一起,形成一个有机的整体。封装是一种信息隐蔽技术,通过封装将对象的私有数据和全体行为的实现隐蔽起来,而将一部分行为(公有)作为对外的接口。从用户或应用人员的角度看,对外的接口就是为其提供的操作功能,即为一组服务,而服务的具体实现,却在对象的内部隐蔽着。在 C++ 中是利用类的形式来实现封装的。

封装性可以使对象的设计者和对象的使用者完全分开,使用者不必知道对象行为的实现细节,只需要使用设计者提供的接口让对象去做。它提高了代码的重用性,降低了对于软件修

改维护的难度。

3. 继承性

继承是面向对象程序设计中最重要特征。继承所表达的是一种对象类之间的相互关系。它使某个类除了具有特有的属性和方法外,还可以继承其他类的属性和方法。在实际工作中,对一个特定问题的认识和处理,前人往往已经进行过较为深入的研究和探讨,他们的研究成果后人可以利用,并且在此基础上有所发展和创造。这正是人类认识发展的过程。

若类之间具有继承关系,则具有以下特征:

- ①类间具有共享特征(包括属性和方法的共享);
- ②类间具有差别或新增部分(包括非共享的属性和方法);
- ③类间具有层次结构。

类的继承机制不仅增强了代码的可重用性,允许程序设计人员在保持原有类的基础上,通过继承进行扩充,从而成为新的类,减少了代码和数据的冗余,提高了软件开发效率,而且减少了模块之间的接口和界面,提高了程序的可维护性。

4. 多态性

多态性也是面向对象的程序设计中的一个重要特征。多态性是指不同的对象接收到相同的消息时,会产生不同的动作。例如,圆形对象和圆柱体对象接收到相同的“求面积”的消息时,它们各自会用不同的方法进行计算:圆形对象计算圆的面积,圆柱体对象计算它的表面积。

C++语言支持两种多态性,即编译时的静态多态性和程序运行时的动态多态性。静态多态性通过函数重载实现,动态多态性通过虚函数机制下的函数重载实现。

5.2 类的定义

类是面向对象程序设计的核心。在C++中,类实际上是一种新的用户自定义的数据类型。在类中不仅包含这类对象的属性,而且包含对该类对象在发生某种事件时的操作。类中的属性称为数据成员,类中的操作称为成员函数。

5.2.1 类的定义格式

一个类主要由两部分组成,即类名和成员。成员包括数据成员和成员函数。成员的访问权限可以是公有、私有和保护。类定义的一般格式如下:

```
class 类名
{
    public:
        公有数据成员和成员函数的定义或说明
    private:
        私有数据成员和成员函数的定义或说明
    protected:
        保护数据成员和成员函数的定义或说明
};
```

未定义的各成员函数的实现

其中, **class** 是定义类所用的关键字。类名是一个标识符。一对花括号内的部分称为类体。注意,花括号之后必须以一个分号“;”结束。关键字 **public**、**private** 和 **protected** 是访问权限说明符。在 **public** 和 **private** 之间说明或定义的成员为公有成员,它们可以被该类对象直接访问。这部分中的成员函数常被称为操作接口。在 **private** 和 **protected** 之间说明或定义的成员为私有成员,它们只能被该类的成员函数直接访问。这部分中的成员函数称为工具函数。在 **protected** 之后说明或定义的成员为保护成员。它们可以像公有成员那样被继承,但又像私有

成员那样仅能被本类的成员函数直接访问。本章仅讨论公有和私有成员。

在类体内, **public**、**private** 和 **protected** 部分出现的顺序是任意的。当私有成员出现在类体开始处时, **private** 可以省略(即缺省访问权限为私有的)。

例 5.1 日期类的定义示例。

```
//成员函数均在类体内定义
#include <iostream.h>
class Date
{
public:
    void SetDate(int y=2006,int m=1,int d=1) //仅对月份进行了正确性检查
    { year=y; month=m>0 && m<13?m:1; day=d;}
    int IsleapYear()
    { return ( year % 400==0)|| ( year % 4==0 && year % 100 !=0 ); }
    int GetYear(){return year;}
    int GetMonth(){return month;}
    int GetDay(){return day;}
    void Print(){ cout<<year<<'. '<<month<<'. '<<day<<endl; }
private:
    int year, month, day;
};
```

例 5.2 学生成绩类的定义示例。

```
//部分成员函数在类体外定义
#include <iostream.h>
#include <iomanip.h>
#include <string.h>
class Student
{ int examstaken; //考试的课程数
  float *marks; //各科考试成绩
public:
    char *name; //学生姓名
    void SetStudent(int e,float *m) { examstaken=e;marks=m;} //设置考试信息
    float averagemark(); //计算平均分函数原型说明
    void format(); //格式化输出函数原型说明
    void Print(); //显示学生信息函数原型说明
};

float Student::averagemark()
{ float total=0;
  for(int i=0;i<examstaken;i++) total+=marks[i];
  return total/examstaken;
}

void Student::format()
{ cout<<setw(10)<<name<<", 平均成绩:"<<setprecision(4)
  <<setw(6)<<averagemark();
}
```

```
void Student::Print()
{ cout<<"学生:"; format();
  cout<<endl;
}
```

从上面例子可以看出,在类的定义中,封装了有关数据和对这些数据的操作。

数据成员必须在类体内说明,而成员函数既可以在类体内定义,也可以在类体外定义。若在类体外定义成员函数,则需在类体内给出它们的原型说明,在类体外的函数名前面加上类名和域作用符“::”,以表示所属的类。

通常情况下,数据成员的访问权限是私有的或保护的,成员函数的访问权限是公有的或保护的;数据成员定义语句的位置既可以在类体最前面,也可以在类体最后面。在例 5.1 中,数据成员的访问权限全是私有的,定义它们的语句放在了类体的最后面。而在例 5.2 中,数据成员的访问权限大部分是私有的,也有一个公有的,但是,这仅仅是为了表示允许出现公有数据成员而已,而不是提倡这样做,定义它们的语句放在了类体的最前面。

5.2.2 定义类时的注意事项

定义类时的注意事项如下。

①在类的定义中,数据成员可以是任何类型。它既可以是基本数据类型,也可以是自定义类型。但无论是什么类型,也无论具有什么访问权限,都不允许对类中数据成员进行初始化;也不允许用存储修饰符 **auto**、**register** 和 **extern** 说明数据成员和成员函数,但允许用存储修饰符 **static** 说明数据成员和成员函数,被说明的数据成员和成员函数称为静态数据成员和静态成员函数。关于静态数据成员和静态成员函数的使用方法将在后面介绍。例如,下面类的定义是错误的:

```
class Date
{ int year(2002);           //错误,不允许对类中数据成员进行初始化
  auto int month;           //错误,不允许用存储修饰符 auto 说明数据成员
  static int day;           //正确,允许用存储修饰符 static 说明数据成员
public:
  static void setDay(int d); //正确,允许用存储修饰符 static 说明成员函数
  extern void print();       //错误,不允许用存储修饰符 extern 说明成员函数
  ...
};
```

②在类体内定义的成员函数将被自动处理为内联函数,而且无需加关键字 **inline**。要使在类体外定义的成员函数成为内联函数,就必须使用 **inline** 关键字。例如,要把前述 **Student** 类类体外定义的成员函数 **format()** 表示为内联函数,必须使用 **inline** 关键字,即

```
inline void Student::format()
{ ... }
```

③不允许本类的变量(称为对象)、数组(称为对象数组)作为本类的成员,但本类的指针或引用可以作为本类的成员。例如:

```
class A
{ int x,y;           //正确
  A a;               //错误
  A *pA;             //正确
:
  ...
};
```

④一个已定义类的对象可以作另一个类的数据成员,这个对象称为对象成员,也有人称

为子对象、嵌入对象、复合对象、组合对象等等。这两个类之间的关系称为复合关系或组合关系,也常常称这种关系为“有”关系(“has a” relation)。例如:

```
#include <iostream.h>

class N{                                //类 N 的定义
    int x;
public:
    void print(){cout<<x<<endl;}
    N(int s=8){x=s;}
};

class M{
    int a;N b;                          //b 是 N 类的对象,是 M 类的对象成员
public:
    void print(){cout<<a<<" "; b.print(); }
    M(int s=4, int t=23):b(t){a=s;}
};
```

对于上面这个例子,可以说 M 类“有一个”N 类的对象 b 作为数据成员,M 类与 N 类具有复合关系。

⑤在开发大型程序时,常将一个类的定义存入一个.h 文件中,而将类体外定义的成员函数存入一个.cpp 文件中,以便灵活地组合、使用。

⑥C++之父 Bjarne Stroustrup 在《C++程序设计语言(特别版)》中这样写到:“构造 class X{...}被称为一个类定义,因为它定义了一个新类型。由于历史的原因,一个类定义也常常被称为一个类声明。”过去在 C 语言中,对“定义”(definition)常常被理解为必须是要求在计算机中分配实际的存储空间,否则称为声明或说明(declaration)。所以,在本书中,更多地称其为一个类定义。

⑦在 C++中,也可以使用关键字 struct 和 union 定义类(详细情况见本章 5.9 节)。

5.3 对象的定义和对象成员的引用

如前所述,类是一种数据类型,这种数据类型的变量称为该类的对象。因此,前面介绍过的关于变量以及数组、指针、引用的许多结论都适用于对象以及对象数组、对象指针、对象引用。例如,对象作形参表示按值传递,对象引用和对象指针作形参表示按地址传递等等。需要注意的是,在 C++程序中对象的使用也是必须先定义后使用。

5.3.1 对象的定义

1. 在定义类的同时定义对象

可以在类体右花括号之后直接给出要定义的对象名。例如:

```
class Date {
public:
    void SetDate(int y=2006,int m=1,int d=1) //仅对月份进行了正确性检查
    { year=y; month=m>0 && m<13?m:1; day=d;}
    int IsleapYear()
    { return ( year % 400==0)|| ( year % 4==0 && year % 100 !=0 ); }
    int GetYear(){return year;}
    int GetMonth(){return month;}
```

```

int GetDay(){return day;}
void Print(){ cout<<year<<'. '<<month<<'. '<<day<<endl; }
private:
    int year, month, day;
}d1,d2;

```

这里的 d1 和 d2 都是 Date 类的对象。

2. 先定义类, 然后再定义对象

在定义了类之后, 定义对象的一般形式为:

类名 对象名表;

或者

class 类名 对象名表;

例如:

Date d1,d2;

或者

class Date d1,d2;

其中的 d1、d2 的意义与前面各对象的意义完全相同。

两种定义对象形式是等效的。使用关键字 class 定义对象的形式是从 C 语言继承下来的, 不使用关键字 class 定义对象的形式才是 C++ 的特色。

因为后一种把类定义与对象定义分开的方法更灵活, 更适于软件重用, 更便于编写大型程序, 所以是更好的方法, 也是提倡的方法。

在定义一个普通对象或对象数组对象时, C++ 编译系统会为每个对象分配存储空间, 以便存放对象中的成员。

3. 在定义对象的同时定义对象数组、对象的引用和指向对象的指针

实际上, 对象名表中不仅可以出现普通对象, 也可以出现对象数组、对象的引用和指向对象的指针等。例如:

Date d1,d[3],*pd=d,*pd1=&d1,&rd1=d1;

这里的 d1 是一个 Date 类的对象; d[3] 是一个 Date 类的长度为 3 的一维对象数组; pd 是指向 Date 类对象的指针, 对其初始化是使其指向对象数组元素 d[0]; pd1 也是指向 Date 类对象的指针, 对其初始化是指向对象 d1; rd1 是一个 Date 类的对象引用, 是对象 d1 的别名。

5. 3. 2 对象成员的引用

类中公有数据成员和成员函数可以被该类的对象直接引用。引用的形式为:

对象名.公有数据成员名

对象名.公有成员函数名(实参表)

对象引用名.公有数据成员名

对象引用名.公有成员函数名(实参表)

对象指针名->公有数据成员名

对象指针名->公有成员函数名(实参表)

这里的“.”和“->”均为成员选择运算符。

例如, 设有定义:

Student s,*sp=&s,&rs=s; (定义对象)

则

s.Print() (使用对象引用公有成员函数 Print)

sp->Print() (使用指针引用公有成员函数 Print)

rs.Print() (通过引用变量引用公有成员函数 Print)

例 5.3 日期类对象的使用示例。

设例 5.1 中的日期类定义已存入 `date.h` 中。

程序如下:

```
#include <iostream.h>
#include "date.h"
void main()
{   Date d1,d2;
    d1.SetDate(2002,2,11); d2.SetDate(2000,10,1);
    if(d2.IsleapYear())   cout<<d2.GetYear()<<"是闰年.\n";
    else cout<<d2.GetYear()<<"不是闰年.\n";
    d1.Print();d2.Print();
    Date *dp=&d2;  dp->SetDate(1949,10,1);
    cout <<"中华人民共和国成立于"<<dp->GetYear()<<"年"
    <<dp->GetMonth()<<"月"<<dp->GetDay()<<"日"<<endl;
}
```

输出结果为:

2000 是闰年.

2002.2.11

2000.10.1

中华人民共和国成立于 1949 年 10 月 1 日

本例中定义了两个 `Date` 类的对象 `d1` 和 `d2`。对它们分别调用成员函数 `SetDate` 而设置私有数据成员 `year`、`month` 和 `day` 的值,使得对象 `d1` 的 `year` 为 2002、`month` 为 2、`day` 为 11,使得对象 `d2` 的 `year` 为 2000、`month` 为 10、`day` 为 1。

`d2.IsleapYear()`判断对象 `d2` 中 `year` 的年份是否为闰年。由于 `d2` 的 `year` 值为 2000,故调用该函数的返回值为 1(真),所以程序执行语句“`cout<<d2.GetYear()<<"是闰年.\n"`”,产生输出结果中的第 1 行。

对象 `d1`、`d2` 调用公有成员函数 `Print()`,显示各自所设置的私有成员 `year`、`month` 和 `day` 的值,产生输出结果的第 2 行和第 3 行。

例中还定义了 `Date` 类的指针 `dp`,并使它指向对象 `d2`。通过指针 `dp` 调用成员函数 `SetDate()`,使 `d2` 对象的各私有成员 `year`、`month` 和 `day` 的值分别置为 1949、10 和 1。因此,当 `cout` 输出时,首先输出字符串“中华人民共和国成立于”,然后通过指针 `dp` 调用成员函数 `GetYear()`、`GetMonth()`和 `GetDay()`,以分别获取 `d2` 对象的 `year`、`month` 和 `day` 的值并相继输出,同时输出“年”、“月”和“日”,产生输出结果的最后一行。

例 5.4 学生成绩类对象的使用示例。

设例 5.2 中的学生成绩类定义已存入 `Student.h` 文件中。

```
#include <iostream.h>
#include "Student.h"
void main()
{   char stname[][10]={"李和平","张民主","白话"};    //3 个学生的姓名
    float stmarks[][5]={{81,82,87,84,85},{78,59,92,79,84},
                        {90,100,89,85,92}};           //3 个学生的成绩
    Student st[3];//定义学生对象数组
    for(int i=0;i<3;i++)                               //操作每个学生对象数组元素
    {   st[i].SetStudent(5,stmarks[i]);                 //设置学生考试信息
```



```

        st[i].name=stname[i];           //设置学生的姓名
        st[i].Print();                  //输出学生的姓名和考试信息
    }
}

```

输出结果为:

```

学生:    李和平,平均成绩:  83.8
学生:    张民主,平均成绩:  78.4
学生:    白话,平均成绩:  91.2

```

注意:name 是 Student 类的公有数据成员,故可直接用表达式

```
st[i].name=stname[i]
```

使数组元素 `st[i]` 的公有数据成员 `name` 取得二维字符数组的第 `i` 行的首地址,即指向该字符串;其次,本例使用了对象数组 `st`,它具有 3 个数组元素,且每个数组元素均为 `Student` 类的对象。这些下标对象与普通对象的使用方式相同。

5.4 对象的初始化

像前边的例子那样定义对象,这些对象的数据成员的初值是不确定的。例如, `Student` 类数组 `st` 的各数组元素的数据成员初值是不确定的,直到执行“`st[i].Setstudent(5, stmarks[i]);`”和“`st[i].name=stname[i];`”两个语句后,对象 `st[i]` 的数据成员才有了确定的值。如果使用了对象的不确定的值,结果是很难想像的。在 C++ 中,可以使用多种方式对对象初始化。

5.4.1 构造函数和析构函数

构造函数和析构函数是类中两种特殊的成员函数。构造函数主要用于为对象分配存储空间和对对象的数据成员进行初始化,而析构函数主要用于执行该函数中的各语句和释放对象所占用的存储空间。由于两个函数有许多共同点,而且两者具有相反的作用,故同时介绍。

1. 构造函数的特点与调用时机

构造函数的一般形式为:

```
构造函数名( ( 形式参数表 ) ) ( : 数据成员初始化表 ) 函数体
```

构造函数有以下特点:

- ①构造函数不允许有任何返回类型;
- ②构造函数名必须与本类的类名相同;
- ③构造函数的参数个数可以为 0,也可以有多个,故构造函数可以重载;
- ④构造函数可以带有一个数据成员初始化表,它必须写在形式参数表的右圆括号之后、函数体的左花括号之前,并且用一个冒号开始。

在数据成员初始化表中对基本类型数据成员的初始化与在构造函数体中用赋值运算传送值的作用完全相同。

例如,日期类的构造函数可以写成:

```
Date(int y=2000,int m=1,int d=1):year(y),month(m),day(d){ }
```

或者写成:

```
Date(int y=2000,int m=1,int d=1){ year=y;month=m;day=d; }
```

请注意,如果用户没有定义任何构造函数,那么,编译程序会自动生成一个无参的、函数体为空的构造函数。显然,由系统生成的这种构造函数只能起到为对象分配存储空间的作用,而不可能起到为对象的数据成员进行初始化的目的。

通常把在调用时不必提供参数的构造函数称为缺省构造函数。因此参数个数为 0 或者全体形参都带有缺省值的构造函数都是缺省的构造函数。

经常遇到的构造函数的调用时机是在程序执行期间,在下列情况下自动调用构造函数:

- ①在遇到对象定义时自动调用构造函数;
- ②在遇到表达式 `new` 类名或 `new` 类名[长度]时自动调用缺省构造函数,遇到表达式 `new` 类名(实参表)时自动调用构造函数;
- ③在遇到表达式 类名(实参表)时自动调用构造函数。

这里所谓的“自动调用构造函数”的意思就是说,构造函数的调用不由用户决定是否调用,而是由编译系统安排的隐式调用。在 C++ 中,这种隐式调用的情况还有很多。因此,这一点给初学 C++ 的人增加了理解上的困难。

例 5.5 带构造函数的日期类示例。

//带构造函数的日期类定义,存入文件 `date1.h` 中

```
#include <iostream.h>
class Date1{
public:
    Date1(); //缺省构造函数
    Date1(int y,int m=1,int d=1); //带部分缺省参数值的构造函数
    void Print();
private:
    int year, month, day;
    int checkDay(int d); //日数正确性检查的工具函数
};
Date1::Date1()
{ year=2000;month=1;day=1;
  cout<<" Constructor without any parameters("<<year<<").\n";
}
Date1::Date1(int y,int m, int d)
{ year=y;
  month=m>0 && m<13 ? m:1; //月份正确性检查
  day=checkDay(d); //日数正确性检查
  cout<<"Constructor with parameters("<<year<<").\n";
}
void Date1::Print() { cout<<year<<". "<<month<<". "<<day; }
int Date1::checkDay( int d )
{ static int Days[13]={0,31,28,31,30,31,30,31,31,30,31,30,31}; //各月正常日数
  if( month !=2 ) //不是 2 月份
  { if( d>0 && d<=Days[month]) return d; }
  else //是 2 月份,区分平年和闰年
  { int d2=( year % 400 == 0 )||( year % 4 == 0 && year % 100 !=0 )? 29:28;
    if( d>0 && d<=d2 ) return d;
  }
  cout<<"日数"<<d<<"不正确,置为 1\n";
  return 1;
}
}
```

其中 `Constructor` 含义是构造函数。

在本例中, `Date1()` 是缺省构造函数, `Date1(int y,int m=1,int d=1)` 是带部分缺省参数值的

构造函数。注意,如果构造函数的全体参数都带有缺省值,该构造函数也是缺省构造函数。例如,可以把本例中的两个构造函数合并为如下的一个缺省构造函数:

```
Date1::Date1(int y=2000,int m=1, int d=1)
{
    year=y; //年份任意
    month=m>0 && m<13 ? m:1; //月份正确性检查
    day=checkDay(d); //日数正确性检查
    cout<<"default constructor ("<<year<<").\n";
}
```

例 5.6 构造函数的使用示例。

```
#include <iostream.h>
#include "date1.h"
void main()
{
    Date1 dt1, dt2(2002),dt3(2004,7,1),dt4(2006,2,29);
    cout<<"default date:";dt1.Print();cout<<endl;
    cout<<"date with default parameters:";
    dt2.Print();cout<<endl;
    cout<<"dt3:"; dt3.Print(); cout<<endl;
    cout<<"dt4:"; dt4.Print(); cout<<endl;
}
```

输出结果为:

```
Constructor without any parameters(2000).
Constructor with parameters(2002).
Constructor with parameters(2004).
日数 29 不正确,置为 1
Constructor with parameters(2006).
default date:2000.1.1
date with default parameters:2002.1.1
dt3:2004.7.1
dt4:2006.2.1
```

在本例中,main()函数开头定义了四个 Date 类的对象。对 dt1,自动调用缺省构造函数进行初始化,其数据成员 year、month、day 分别为 2000、1 和 1;对 dt2,自动调用带部分缺省参数值的构造函数进行初始化,其数据成员 year、month、day 分别为 2002、1 和 1;对 dt3,也是自动调用带部分缺省参数值的构造函数进行初始化,其数据成员 year、month、day 分别为 2004、7 和 1;对 dt4,也是自动调用带部分缺省参数值的构造函数进行初始化,在检查日数时发现错误而置为 1,其数据成员 year、month、day 分别为 2006、2 和 1。

从输出结果可见,在遇到定义对象的语句时,系统会按照各对象书写的顺序给各对象分配存储空间并同时调用相应构造函数对其进行初始化。

2. 析构函数的特点与调用时机

(1)特点

析构函数有如下特点:

- ①析构函数名为本类的类名前面加一个“~”符号;
- ②析构函数不允许带有任何参数,故析构函数一个类中只有一个且不可能重载;
- ③析构函数不允许有任何返回类型。

(2)析构函数的调用时机

在程序执行期间,遇到下列情况时自动调用析构函数:

- ①在遇到复合语句执行结束、函数执行结束或程序执行结束时调用析构函数,释放复合语句、函数或整个程序中的局部对象或全局对象所占用的存储空间;
- ②在执行表达式 `delete` 指向对象的指针或 `delete[]` 指向对象数组的指针时调用析构函数,释放大用 `new` 运算所动态分配的对象所占用的存储空间。

如果用户没有定义析构函数,则编译程序自动生成一个函数体为空的析构函数。这种析构函数可以释放除了用 `new` 运算所分配的任何对象的存储空间。所以,一般情况下不需要显式地定义析构函数。但是,若不使用析构函数释放大用 `new` 运算所分配的“动态”对象所占用的存储空间,造成这些存储空间不能被重新分配和不能被任何对象引用的问题(即内存泄漏问题),不仅会造成存储空间的浪费,而且可能引起程序执行异常。也就是说,当类中包含用 `new` 运算分配存储空间的活动时,必须给出析构函数的明确定义,并用 `delete` 运算释放这些存储空间。

例 5.7 带析构函数的日期类示例。

//带析构函数的类 `Date2` 的定义,存入文件 `date2.h` 中

```
#include <iostream.h>
class Date2 {
public:
    Date2(int y=2000,int m=1,int d=1); //全体参数都带缺省值的构造函数
    ~Date2();                          //析构函数
    void Print();
private:
    int year, month, day;
};
Date2::Date2(int y,int m, int d):year(y),month(m),day(d){ } //缺省构造函数
Date2::~~Date2()    //析构函数
{ Print();
  cout<<" Destructed.\n";
}
void Date2::Print() { cout<<year<<". "<<month<<". "<<day; }
```

本例中 `Destructor` 的含义是析构函数。

例 5.8 析构函数使用示例。

```
#include <iostream.h>
#include "date2.h"
void main()
{ Date2 dt1, dt2(2006),dt3(1921,7,1);
  dt1.Print();cout<<endl;
  dt2.Print();cout<<endl;
  dt3.Print();cout<<endl;
}
```

输出结果为:

```
2000.1.1
2006.1.1
1921.7.1
1921.7.1 Destructed.
```

2006.1.1 Destructed.

2000.1.1 Destructed.

在该程序执行时,首先对三个 `Date` 类的对象 `dt1`、`dt2` 和 `dt3` 分别自动调用缺省构造函数进行初始化。然后,分别输出 `dt1`、`dt2` 和 `dt3` 的年份、月份和日数的值。在遇到主函数体的右花括号时,程序即将执行结束。在程序执行结束之前,系统对对象 `dt3`、`dt2` 和 `dt1` 分别自动调用析构函数,执行该函数中的各语句后,释放各对象所占用的存储空间。

从程序输出结果的最后三行可以看出,析构函数调用的顺序通常是与构造函数的调用顺序相反的。

例 5.9 构造函数和析构函数的调用顺序示例。

```
#include <iostream.h>
#include <math.h>
//构造函数与析构函数调用顺序示例
class Point{ //点 Point 类的定义
    double x, y;
public:
    Point(double a=0,double b=0) //构造函数
    { x=a; y=b; cout<<"构造点("<<x<<","<<y<<")\n"; }
    void Print(){ cout<<"点("<<x<<","<<y<<") "; } //显示点信息函数
    double Distance( Point &pr ) //计算两点间的距离函数
    { double dx=x-pr.x, dy=y-pr.y;
      return sqrt(dx*dx+dy*dy);
    }
    ~Point(){ cout<<"析构点("<<x<<","<<y<<")\n"; } //析构函数
};
void main()
{ Point ob1, ob2(-3,4);
  ob1.Print(); cout<<"到"; ob2.Print(); cout<<"的距离=";
  cout<<ob1.Distance( ob2 );
  cout<<endl;
}
```

输出结果为:

```
构造点(0,0)
构造点(-3,4)
点(0,0) 到点(-3,4) 的距离=5
析构点(-3,4)
析构点(0,0)
```

要注意,对象的生存期会改变析构函数的调用顺序。

全局对象、静态对象和局部对象有不同的生存期。它们生存期的规定分别与全局变量、静态变量和局部变量的生存期规定相同。

全局对象和全局静态对象的生存期与所在程序的执行期间相同。

局部静态对象的生存期自程序第一次执行到它们所在的程序块、定义它们的语句开始,直到整个程序执行结束为止。

局部对象的生存期自程序第一次执行到它们所在的程序块、定义它们的语句开始,直到所在的程序块执行结束为止。

例 5.10 对象生存期中各对象构造函数和析构函数的调用顺序示例。

```
#include <iostream.h>
#include <string.h>
class Test {
public:
    Test(char *s)                //构造函数
    { strcpy(string,s);
      cout<<"对"<<string<<"调用构造函数.\n";
    }
    ~Test(){ cout<<"对"<<string<<"调用析构函数.\n"; } //析构函数
private:
    char string[80];
};
Test GlobalObject1("Global Object1");           //全局对象 1
static Test GlobalStaticObject("Global Static Object");//全局静态对象
void fun()//外部函数
{ Test FunObject("Object of fun"); //局部对象
  static Test FunStaticObject("Static Object of fun");//外部函数局部静态对象
  cout<<"在 fun()中.\n";
}
void main() //主函数
{ Test MainObject1("Object1 of main");           //主函数局部对象 1
  static Test MainStaticObject("Static Object of main");//主函数局部静态对象
  cout<<"调用 fun()之前,在 main()中.\n";
  fun();
  Test MainObject2("Object2 of main");           //主函数局部对象 2
  cout<<"调用 fun()之后,在 main()中.\n";
}
Test GlobalObject2("Global Object2");           //全局对象 2
```

输出结果为:

```
对 Global Object1 调用构造函数.
对 Global Static Object 调用构造函数.
对 Global Object2 调用构造函数.
对 Object1 of main 调用构造函数.
对 Static Object of main 调用构造函数.
调用 fun()之前,在 main()中.
对 Object of fun 调用构造函数.
对 Static Object of fun 调用构造函数.
在 fun()中.
对 Object of fun 调用析构函数.
对 Object2 of main 调用构造函数.
调用 fun()之后,在 main()中.
对 Object2 of main 调用析构函数.
对 Object1 of main 调用析构函数.
```

对 Static Object of fun 调用析构函数。
 对 Static Object of main 调用析构函数。
 对 Global Object2 调用析构函数。
 对 Global Static Object 调用析构函数。
 对 Global Object1 调用析构函数。

程序说明如下。

①程序运行开始时,首先分别调用构造函数相继创建全局对象 GlobalObject1、全局静态对象 GlobalStaticObject 和全局对象 GlobalObject2,显示如下信息:

对 Global Object1 调用构造函数。
 对 Global Static Object 调用构造函数。
 对 Global Object2 调用构造函数。

②然后进入 main()函数,先后调用构造函数相继创建局部对象 MainObject1、局部静态对象 MainStaticObject,显示如下信息:

对 Object1 of main 调用构造函数。
 对 Static Object of main 调用构造函数。

③接着执行主函数中头一个 cout 语句,显示如下信息:
 调用 fun()之前,在 main()中。

④接着执行主函数中调用 fun()外部函数的语句,在 fun()函数中先后创建局部对象 FunObject 和局部静态对象 FunStaticObject,相继调用构造函数,显示如下信息:

对 Object of fun 调用构造函数。
 对 Static Object of fun 调用构造函数。

又在 fun()函数中执行 cout 语句,显示如下信息:
 在 fun()中。

⑤在 fun()函数执行结束时,释放局部对象 FunObject,调用析构函数,显示如下信息:
 对 Object of fun 调用析构函数。

⑥从 fun()返回到 main()函数后,创建局部对象 MainObject2,调用构造函数,显示如下信息:

对 Object2 of main 调用构造函数。

接着执行主函数中的第2条 cout 语句,显示如下信息:
 调用 fun()之后,在 main 中。

⑦main()函数结束时,先后调用析构函数释放局部对象 MainObject2 和 MainObject1,显示如下信息:

对 Object2 of main 调用析构函数。
 对 Object1 of main 调用析构函数。

⑧整个程序结束前,先后调用析构函数释放局部静态对象 FunStaticObject 和 MainStaticObject,以及释放全局对象 GlobalObject2、全局静态对象 GlobalStaticObject 和全局对象 GlobalObject1,显示如下信息:

对 Static Object of fun 调用析构函数。
 对 Static Object of main 调用析构函数。
 对 Global Object2 调用析构函数。
 对 Global Static Object 调用析构函数。
 对 Global Object1 调用析构函数。

请读者根据以上输出结果和说明,体会各种对象的生存期和被创建、释放的时机。

5.4.2 拷贝构造函数

在C++中,如果ob1和ob2是两个同一类的对象,那么,表达式ob2=ob1的默认含义就是将对象ob1的全体成员逐个拷贝(即赋值)给对象ob2。在一般情况下,这样的结果确实是需要。但是,对于在构造函数和析构函数中含有资源管理的对象(如含有指针成员、文件成员等的类的对象)来说,就可能产生意外的问题。例如:

```
class Array {
    int *buf,size;
public:
    Array(int n=10) { size=n>0?n:10; buf=new int[size]; }
    ~Array(){ delete []buf; }
    ... : //其他成员函数
};
void h()
{ Array a1(15), a2=a1, a3;
  a3=a2;
}
```

在函数h()中,Array类的缺省构造函数为a1和a3,各调用了一次,共二次,而Array类的析构函数却被调用了三次,即对a1、a2和a3各调用了一次。另外,因为赋值的默认含义是按成员复制,所以,在函数h()结束时,a1、a2和a3中的指针成员都指向创建a1时所分配的数组存储空间,而在指向创建a3时所分配的数组存储空间的指针被赋值a3=a2覆盖了。这样,该数组存储空间可能被永远丢掉了。此外,由于为创建a1而分配的数组存储空间同时被a1、a2和a3中的指针成员指向,它将被释放三次。在释放a3时,该数组存储空间已经被释放了,从而造成a2和a1中的指针成员的指向失去定义(这种现象有人称为指针悬挂),因此可能产生灾难性的后果。

为了避免上述情况的发生,就必须引入拷贝构造函数和对自定义的类明确赋值运算的含义(即重载赋值运算符)。此处仅仅讨论拷贝构造函数。

拷贝构造函数是一种特殊的成员函数,作用是用一个已定义的对象初始化一个被创建的同类对象。

拷贝构造函数具有如下特点:

- ①该函数名与本类的类名相同,且不允许指定返回类型,故它是一个构造函数;
- ②该函数只有一个参数,而且必须是一个本类对象的引用。

请注意,如果用户没有定义拷贝构造函数,那么,编译程序会自动生成一个实现按成员复制的拷贝构造函数。显然,由系统生成的这种拷贝构造函数可能会造成指针悬挂、内存泄漏等灾难性的后果。

在程序执行期间,遇到下列情况时自动调用拷贝构造函数:

- ①在遇到用一个已定义的对象初始化一个新的同类对象时调用拷贝构造函数;
- ②在调用一个函数时,要把实参对象按值传递给相应形参对象需要调用拷贝构造函数;
- ③在遇到把对象作为函数返回值时调用拷贝构造函数。

下面首先看几个使用拷贝构造函数的简单例子。

例5.11 拷贝构造函数使用示例1。

```
//调用拷贝构造函数,用一个已定义的对象初始化一个新的同类对象
#include <iostream.h>
class Point
{ public:
```



```

    Point(){x=0.0;y=0.0;}
    Point(double x1,double y1) {x=x1;y=y1;cout<<"构造函数被调用.\n";}
    Point(Point &p); //拷贝构造函数说明
    double Xcoord(){ return x; }
    double Ycoord(){ return y; }
private:
    double x,y;
};
Point::Point(Point &p) //拷贝构造函数的实现
{
    x=p.x; y=p.y;
    cout<<"拷贝构造函数被调用.\n";
}
void main()
{
    Point p1(5.5,7.7),p2(p1),p3;
    cout<<"p1=("<<p1.Xcoord()<<","<<p1.Ycoord()<<")\n"
        <<"p2=("<<p2.Xcoord()<<","<<p2.Ycoord()<<")\n"
        <<"p3=("<<p3.Xcoord()<<","<<p3.Ycoord()<<")\n";
    p3=p1; //对象赋值语句
    cout<<"p3=("<<p3.Xcoord()<<","<<p3.Ycoord()<<")\n";
}

```

输出结果为:

```

构造函数被调用.
拷贝构造函数被调用.
p1=(5.5,7.7)
p2=(5.5,7.7)
p3=(0,0)
p3=(5.5,7.7)

```

程序说明如下。

①进入 main()函数时,首先遇到定义 Point 类对象的语句,对于对象 p1(5.5,7.7),调用构造函数对 p1 初始化,使 p1 的 x 值为 5.5,y 值为 7.7,并显示如下信息:

构造函数被调用.

接着遇到定义 Point 类的对象 p2(p1)。由于 p1 为已知的对象,故以 p1 为实参、调用 Point 类的拷贝构造函数,用 p1 的数据成员值对 p2 进行初始化,并显示如下信息:

拷贝构造函数被调用.

接着遇到定义 Point 类的对象 p3,调用缺省构造函数对 p3 初始化,使数据成员 x 和 y 的值均为 0。特别要指出的是,定义 Point 类的对象的语句中的 p2(p1),完全可以写为 p2=p1,但是,这里的“=”不是赋值运算符,而是初始化记号。所以,不能理解成把 p1 的数据成员逐个复制给 p2,而要理解成以 p1 为实参调用拷贝构造函数,为 p2 初始化。

②当遇到赋值表达式 p3=p1 时,由于 C++ 已经扩展了赋值运算的功能,当赋值运算符两端均为同类对象时,它把赋值运算符右端的对象 p1 的数据成员逐个复制给赋值运算符左端的同类对象 p3 的相应数据成员。所以,p3 的 x 值改为 5.5,y 值改为 7.7。当然,在类 Point 中的数据成员不包含指针,它的构造函数也没有分配存储空间等动态功能,因此,在这种情况下同类对象之间的赋值是安全的。

例 5.12 拷贝构造函数使用示例 2。

```

//调用拷贝构造函数把实参对象按值传递给相应形参对象
//调用拷贝构造函数把对象作为函数返回值
#include <iostream.h>
class Point
{ public:
    Point(){x=0.0;y=0.0;}
    Point(double x1,double y1) {x=x1;y=y1;cout<<"构造函数被调用.\n";}
    Point(Point &p); //拷贝构造函数说明
    double Xcoord(){ return x; }
    double Ycoord(){ return y; }
private:
    double x,y;
};
Point::Point(Point &p)//拷贝构造函数的实现
{ x=p.x; y=p.y;
  cout<<"拷贝构造函数被调用.\n";
}
Point pf(Point Q)//对象作函数参数
{ cout<<"进入 pf()函数.\n";
  double x,y;
  x=Q.Xcoord()+10;y=Q.Ycoord()+20;
  Point R(x,y);
  return R;//返回对象的值
}
void main()
{ Point p1(5,7),p2;
  p2=pf(p1);
  cout<<"p2=("<<p2.Xcoord()<<","<<p2.Ycoord()<<")\n";
}

```

输出结果为:

```

构造函数被调用.
拷贝构造函数被调用.
进入 pf()函数.
构造函数被调用.
拷贝构造函数被调用.
p2=(15,27)

```

程序说明如下:

①在调用 `pf(p1)` 函数时,实参对象 `p1` 与对应形参对象 `Q` 相结合,此时要调用拷贝构造函数,用 `p1` 给 `Q` 初始化,故有输出结果的第 2 行信息;

②在 `pf()` 函数中,遇到定义 `Point` 类的对象 `R` 的语句时调用构造函数,故有输出结果的第 4 行信息;

③当遇到 `pf()` 函数中的 `return R` 语句时,系统调用拷贝构造函数,用 `R` 初始化一个匿名对象,故有输出结果的第 5 行信息。

读者可以从上面的例子体会调用拷贝构造函数的不同情况。而且知道了,与变量一样,

对象、对象的引用等可以作为函数的形式参数。

下面再看一个稍微复杂的必须使用拷贝构造函数的例子。

例 5.13 必须使用拷贝构造函数的例子——一维整数数组类。

```
#include <iostream.h>
class intArray {
    int *buf; //指向一维整数数组的指针
    int size; //一维整数数组的长度
public:
    intArray(int n=10) //构造函数
    {   size=n>0?n:10;
        buf=new int[size];
        cout<<"输入"<<size<<"个整数:";
        for(int k=0; k<size; k++) //通过输入数据初始化一维整数数组
            cin>>buf[k];
    }
    ~intArray(){ delete []buf; } //析构函数不可缺少,否则可能造成“内存泄漏”
    intArray( intArray &r ) //拷贝构造函数不可缺少,否则可能造成“指针悬挂”
    {   size=r.size; //设置当前对象的数组长度
        buf=new int[size]; //为当前对象的数组分配内存空间
        for(int k=0; k<size; k++) //把对象 r 的数组元素值赋给当前对象
            buf[k]=r.buf[k];
    }
    void assign( intArray &r ) //实现与赋值等效的功能
    {   delete []buf; //释放当前对象的数组内存空间
        size=r.size; //重新设置当前对象的数组长度
        buf=new int[size]; //重新为当前对象的数组分配内存空间
        for(int k=0; k<size; k++) //把对象 r 的数组元素值赋给当前对象
            buf[k]=r.buf[k];
    }
    void print() //显示当前对象的数组中各数组元素值
    {   for(int k=0; k<size; k++) cout<<buf[k]<<" ";
        cout<<endl;
    }
};

void main()
{   intArray a1(5);
    cout<<"a1:"; a1.print();
    intArray a2;
    cout<<"a2:"; a2.print();
    intArray a3=a1;
    cout<<"a3:"; a3.print();
    a3.assign(a2);
    cout<<"在调用 assign(a2)后的 a3:"; a3.print();
}
```

输出结果为:

```

输入 5 个整数:1 2 3 4 5
a1:1 2 3 4 5
输入 10 个整数:0 1 2 3 4 5 6 7 8 9
a2:0 1 2 3 4 5 6 7 8 9
a3:1 2 3 4 5
在调用 assign(a2)后的 a3:0 1 2 3 4 5 6 7 8 9

```

5.4.3 子对象的初始化

如果一个类含有子对象,创建对象时不仅要对本类的基本类型数据成员初始化,而且要对子对象进行初始化。此时构造函数的形式常常是:

构造函数名(形参表):数据成员初始化表 函数体

其中,数据成员初始化表的形式为:

子对象初始化表[,本类基本类型数据成员初始化表]

当在程序中创建具有子对象的类的对象时,将调用相应类的构造函数,并且首先按照各子对象的定义顺序(不按照各子对象在子对象初始化表中的顺序)调用它们各自所属类的构造函数对各子对象初始化,然后再执行对本类基本类型数据成员初始化,并执行本类的构造函数体。

如果创建对象时调用的是缺省构造函数,则对子对象的初始化也是调用相应类的缺省构造函数。

特别要注意,如果在初始化表中没有包含对子对象初始化的部分,则自动调用子对象相应类的缺省构造函数对各子对象进行初始化。

当释放具有子对象的类的对象时,先按本类析构函数释放本类数据成员,再按初始化时的相反顺序调用各子对象所属类的析构函数释放各子对象。

例 5.15 子对象初始化示例。

```

#include <iostream.h>
class B
{ public:
    B(){a=2;b=5; cout<<"B class(a=2,b=5)\n"; }
    B(int i,int j) {a=i;b=j;cout<<"B class(a="<<a<<","b="<<b<<")\n"; }
    void print(){cout<<" a="<<a<<" b="<<b<<endl; }
private:
    int a,b;
};
class A
{ public:
    A(){x=0; cout<<"A class(x=0)\n"; }
    A(int i,int j, int k):bb(i,j),x(k) { cout<<"A class(x="<<x<<")\n"; }
    void print() { cout<<"x="<<x; bb.print(); }
private:
    int x;
    B bb;    //A 类中包含 B 类的对象作私有数据成员,即子对象
};
void main()

```

```

{ A m,n(3,9,7);
  cout<<"m:"; m.print();
  cout<<"n:"; n.print();
}

```

输出结果为:

```

B class(a=2,b=5)
A class(x=0)
B class(a=3,b=9)
A class(x=7)
m:x=0 a=2 b=5
n:x=7 a=3 b=9

```

程序说明如下:

①当遇到主函数中类 A 对象 m 的定义时,由于没有给出初始化参数,所以调用类 A 的缺省构造函数对 m 初始化。按照上面的说明,首先应该执行类 B 的缺省构造函数,对 m 中所包含的子对象 bb 初始化,从显示结果的第 1 行可以验证这一点。从类 B 的缺省构造函数返回后接着执行类 A 的缺省构造函数,把 m 所包含的整型变量 x 赋值为 0,并输出第 2 行,并返回到主函数。

②接着遇到主函数中类 A 对象 n 的定义。由于给出初始化参数,所以调用类 A 的带参构造函数对 n 初始化。按照上面的说明,首先应该执行类 B 的带参构造函数,对 n 中所包含的子对象 bb 初始化,从显示结果的第 3 行可以验证这一点。从类 B 的带参构造函数返回后接着把类 A 数据成员 x 初始化为 k(此处 k 为 7)。最后执行类 A 的带参构造函数的函数体,显示输出结果的第 4 行信息。

③类 A 和类 B 均有成员函数 print(),但编译程序会根据当前对象所属的类调用相应类的成员函数,此点称为类作用域。每一个类都具有该类的类作用域。该类的数据成员和成员函数的作用域都局限于该类所属的类作用域中。

④C++规定,在类作用域中定义的数据成员不能使用 auto、register 和 extern 等存储修饰符,但可以用 static 修饰符;在类作用域中定义的成员函数也不能使用 extern 修饰符,但可以用 static 修饰符。使用 static 修饰符的作用将在第 6 章中介绍。

⑤一般来说,类作用域介于文件作用域和块作用域之间。但其情况比较复杂,只能根据具体问题具体分析。此处不作详细讨论。

⑥本类基本类型数据成员的初始化既可写入初始化表中,也可写到构造函数体内。例如,类 B 构造函数可写成:

```

B(int i,int j):a(i),b(j) { cout<<"B class(a="<<a<<",b="<<b<<")\n"; }

```

其效果与例中的写法等价。显然,像本例这样把本类基本类型数据成员的初始化写入初始化表中,再去掉其中的 cout 语句,则函数体仅为一对花括号。

5.5 this 指针

this 是一个隐含于每个类的成员函数中的特殊指针(包括构造函数、析构函数等全体类的成员函数,但不包括静态成员函数),它指向当前调用成员函数的对象(以下简称为当前对象)。

当一个对象调用成员函数时,成员函数除了接收指定的实参外,同时还接收了该对象的地址。这个地址被一个隐含的形参 this 指针所获取,它等同于执行“this=该对象的地址”,然

后才调用成员函数。而且所有对数据成员的访问都隐含地被加上前缀 **this->**。因此,成员函数存取数据成员时,都隐含使用 **this** 指针。**C++**正是靠 **this** 指针的当前值来感知当前对象的。这就是在成员函数中访问数据成员时无须对象名作前缀的原因。

在一般情况下,并不需要明确地使用 **this** 指针来引用数据成员。既然 **this** 指向当前对象,那么, ***this** 就表示当前对象。

例 5.16 **this** 和 ***this** 的使用示例。

```
#include <iostream.h>
class Test
{ public:
    Test(){m=10;n=5;}
    Test & setm (int);
    Test & setn(int);
    void show();
private:
    int m,n;
};
void Test::show() { cout<<"m="<<this->m<<"n="<<(*this).n<<endl;}
Test & Test::setm(int i){ m=i;return *this; }
Test & Test::setn(int j){ n=j;return *this; }
void main()
{ Test x;
  x.show();
  x.setm(5).setn(3).setm(9);
  x.show();
}
```

输出结果为:

m=10,n=5

m=9,n=3

前面曾经提到,成员函数在存取数据成员时,都隐含地使用了 **this** 指针。本例中的 **Test** 类的构造函数 **Test(){m=10;n=5;}**就是这样。也可以把它写成下面显式使用 **this** 指针的形式:

```
Test(){this->m=10; this->n=5;}
```

本例除了表明 **this** 指针的用法外,同时还说明了返回对象引用的函数 **setm()**和 **setn()**的用法。由于这两个函数都是返回当前对象(**return *this**)本身,所以,程序中的 **x.setm(5)**除了设置对象 **x** 的数据成员 **m** 为 5 外,还返回对象 **x** 本身。在调用成员函数 **setm()**返回后,使得 **setm(5).setn(3)**相当于 **x.setn(3)**。这样,在调用函数 **setn()**期间将设置对象 **x** 的数据成员 **n** 为 3,同时返回对象 **x** 本身。在调用成员函数 **setn()**返回后,使得 **setm(5).setn(3).setm(9)**相当于 **x.setm(9)**,又重新设置对象 **x** 的数据成员 **m** 为 9。

正是由于返回对象引用,才能够连续调用成员函数 **setm(5).setn(3).setm(9)**。这是返回对象引用的好处之一。

5.6 其他定义类的形式

struct 和 **union** 原来是 C 语言中用户定义结构类型和联合类型的关键字。在 **C++**中,也可以使用关键字 **struct** 和 **union** 定义类。下面分别介绍用关键字 **struct** 和 **union** 定义类的方

法,并简单介绍关于结构类型和联合类型。

5.6.1 用关键字 struct 定义类

用 struct 定义类时,类体内容和格式与用 class 定义类完全相同。它们的差别是:用 struct 定义类时缺省的访问权限是公有的(public),而用 class 定义类时缺省的访问权限是私有的(private)。用 struct 定义的类的对象使用方法与用 class 定义的类的对象使用方法完全相同。

例 5.17 用 struct 定义日期类 Date。

```
#include <iostream.h>
struct Date
{
    //公有成员函数,不必使用标号 public
    Date(int =1900,int =1,int =1); //构造函数
    void print() //按“XXXX 年 X 月 X 日”格式输出日期
    { cout<<year<<"年"<<month<<"月"<<day<<"日\n"; }
private:
    //私有数据成员和成员函数
    int year; //任意年份
    int month; //1~12
    int day; //1~31(与月份有关)
    int checkDay(int); //根据年份、月份检查日数的工具函数
};
//构造函数:检查月份的正确性,并调用工具函数
Date::Date(int y, int m, int d)
{ year=y;
  month=(m>0 && m<13)?m:1;
  day=checkDay(d);
}
//工具函数,根据年、月值检查日数的正确性
int Date::checkDay(int d)
{ static int daysPerMonth[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
  if(month!=2) { if(d>0 && d<=daysPerMonth[month]) return d; } //不是 2 月
  else //是 2 月,检查是否为闰年
  { int days=((year%4==0 && year%100!=0)||year%400==0)?29:28;
    if( d>0 && d<=days) return d;
  }
  cout<<"错误日数值"<<d<<"已被设置为 1.\n";
  return 1; //当给出时,使对象处于稳定状态
}
void main()
{ Date d1(2006,5,21),d2(1967,4,31);
  d1.print(); d2.print();
}
```

输出结果为:

2006 年 5 月 21 日

错误日数值 31 已被设置为 1。

1967 年 4 月 1 日

5.6.2 结构及其使用简介

在 C 语言中,常常用 **struct** 定义“结构类型”,简称为结构或结构体。

结构是由用户自己定义的用其他类型的元素建立的复合数据类型。在程序中使用结构时,必须首先定义。例如,描述学生情况的结构可定义为:

```
struct Student {
    int no;           //学号
    char name[15];    //姓名
    char sex[3];      //性别
    float score[5];   //5 门课程的成绩
};
```

可见,定义结构时用关键字 **struct** 引入,这是结构类型定义的标志。**Student** 称为结构类型名,由用户命名,简称为结构名或结构体名,在 C++ 中作为新的类型名使用。由一对 **{}** 组成的是结构体。在结构体中说明的变量 **no**、**name** 等称为结构的成员,对每个成员都要指明它们的类型和名字。每个成员的类型既可以是基本数据类型,也可以是指针或数组、结构、联合等复合数据类型。本结构的变量、数组不能作为自己的成员,但是允许本结构的指针作为自己的成员,当包含本结构的指针作为自己的成员时,称为自引用结构。使用自引用结构可以形成链接数据结构,如链表、队列、堆栈和树等(见本书后面的数据结构部分)。每个成员名后用“;”结束。显然,同一结构的成员应有唯一的名称。但是两个不同结构可以包含同名的成员不会发生冲突。结构体后也用“;”作为结构定义的结束。

定义结构类型后,在程序中就可以说明结构型变量、引用或指针,并且可以在说明它们的同时初始化,如

```
struct Student s1={301127,"李和平","男",{87.5,83.2,79.8,90.6,94.0}},&rs=s1,*ps=&s1;
或
```

```
Student s1={301127,"李和平","男",{87.5,83.2,79.8,90.6,94.0}},&rs=s1,*ps=&s1;
```

编译系统在处理结构类型时,为结构变量中的每个成员都分配相应的存储单元。结构变量占用的内存空间至少为全体成员空间之和,而且不同的编译系统为相同的结构类型变量所分配的存储空间可能不同。

在程序中使用成员访问运算符“.”和“->”访问结构成员。例如,对于结构 **Student**, 下面的变量 **s1**、引用 **rs** 和指针 **ps** 引用成员的形式都是正确的:

```
s1.no    s1.name    rs.no    rs.name    ps->name    ps->score[2]
```

例 5.17 学生结构使用示例。

```
#include <iostream.h>
struct student
{
    int no;
    char name[10];
    float score;
}a[3]={ {1001,"李和平",85.5},{1002,"王华",90.0},{1003,"张民主",78.5} };
float aver(float a,float b,float c){ return (a+b+c)/3; } //计算平均分函数
void main( )
{
    int i;
    student *p=a;
    for (i=0;i<3;i++, p++)    cout<<p->no<<"          "<<p->name<<"
"<<p->score<<endl;
    cout<<"平均分="<<aver(a[1].score, a[2].score, a[3].score)<<endl;
}
```


程序执行结果:

```
1001  李和平  85.5
1002  王华   90
1003  张民主  78.5
平均分=56.1667
```

从上面的例子可以再一次地看出,C++非面向对象的程序是由一个主函数和多个其他(外部)函数组成的。所以,使用非面向对象的方法设计程序的重点是设计函数或过程实现在某些数据结构上的操作或算法,通过有机地组织这些函数或过程的相互调用达到预期的目的。数据往往是作为函数或过程的相互调用中传递的参数。因此,数据与对它们的操作是相互独立、相互分离的。这样设计的程序难以修改和重用。所以,非面向对象的方法较适于设计小规模、短期使用的程序。

而使用面向对象的方法设计程序,重点是设计被处理的类和各个相关类之间的关系,通过各类的对象调用本类的成员函数达到预期的目的。在类的定义中,不仅包含自己的数据,而且包含对这些数据的操作。因此,数据与对它们的操作是密切相关和相互依存的。这样设计的程序便于修改和重用。显然,面向对象的方法更适于设计大规模、长期使用的程序。

结构(体)也可以用类实现,只需要用 **public** 关键字将成员表示为公有即可。

5.6.3 联合和用关键字 union 定义类

1. 联合

联合也是一种由用户定义的数据类型,又称为联合体或共用体。它在定义和使用形式上与结构类型相似,只是使用内存的方式不同。

联合在使用之前也要先定义。联合的定义形式为:

```
union 联合名{
    数据类型 成员 1;
    数据类型 成员 2;
    ...
    数据类型 成员 n;
};
```

例如,定义一个名为 **tag** 的联合类型如下:

```
union tag{
    int ai;
    float af;
    char ac;
};
```

联合类型定义后,可以说明联合类型的变量或指针。说明联合类型变量的方法与结构相同,可以在定义联合类型的同时说明联合类型的变量,也可以先定义,另外说明。例如:

```
union tag{
    int ai;
    float af;
    char ac;
}x,y,*p;
```

或者

```
union tag{
    int ai;
    float af;
```

```
char ac;
};
union tag x,y,*p;
```

在 C++ 中可以写成“tag x,y,*p;”。

联合成员的引用方式与结构的引用完全相同。以下引用都是正确的:

```
x.ai    x.af    x.ac    p->ai    p->af    p->ac
```

联合与结构的区别是:编译系统为结构变量中的每个成员都分配相应的存储单元,结构变量占用的内存空间至少为全体成员空间之和;而联合类型的变量不管有多少成员,编译系统为联合分配一个能足以容纳其中需要存储字节个数最大的成员的存储空间,各成员共用同一个存储空间。因此,在同一时间内,只能存放一个成员,且总是以后者取代前者。联合提供了单个区域内存存储不同类型变量的方法,可以在不同的时间内维持不同类型的数据。使用联合的主要目的是节省存储空间。不同的编译系统为相同的联合类型变量所分配的存储空间可能不同。

使用联合需注意以下几点:

- ①联合不能在定义时初始化;
- ②除了两个相同联合类型的变量可以直接进行赋值外,不能直接存取联合变量,只能对联合成员进行操作;
- ③联合成员的地址即为联合变量的地址;
- ④联合可以出现在结构或数组中,结构或数组也可以出现在联合体内。

例 5.19 联合使用举例。

```
#include <iostream.h>
union Num{
    int a;
    float f;
};
void main()
{   Num x;
    x.a=15;
    cout<<"x.a="<<x.a<<endl;
    x.f=3.14159;
    cout<<"x.f="<<x.f<<endl;
    cout<<"x.a="<<x.a<<endl;
}
```

输出结果为:

```
x.a=15
x.f=3.14159
x.a=1078530000
```

从结果的后两行可以看出,float 类型与 int 类型的值在计算机内部的表示差别很大,所以在给其中一个联合成员赋值时,就使得不同内部表示的联合成员失去了意义。

误用联合将产生非常微妙的错误,所以应尽可能地避免使用联合。

2. 用关键字 union 定义类

用 union 定义类时,类体内容与格式虽与用 class 定义类相同,但缺省访问权限是公有的,而且,在任何时候只能使用其中一个数据成员。可以用构造函数初始化任何数据成员。对于联合体类还有其他很多限制。关于 union 定义类的详细情况本书不作讨论。

5.6.4 类型定义语句 typedef

在程序设计中,除了可以使用系统提供的基本数据类型名和用户自定义的数据类型名外,还可以用类型定义语句 **typedef** 为已有的数据类型定义新的类型名。

类型定义的一般形式是:

typedef 已有类型名 新类型名;

例如,可以对频繁使用而拼写较长的类型名定义一个同义词:

typedef unsigned int uint;

其中的 **uint** 就是 **unsigned int** 的新名。在此说明以后,程序中就可以用新类型名字定义变量:

uint ua,ub;

typedef 的另一个用法是将对某个类型的直接引用限制到一个地方。例如,

typedef int int32;

typedef short int int16;

而且,这样的程序可以很容易地移植到一个 **sizeof(int)** 为 2 的环境上,只需要在代码中重新定义 **int32**, 即

typedef long int32;

需要说明的是:类型定义 **typedef** 语句并不产生新的数据类型,也不定义存储单元,只是给已有的类型增加新的类型名,为编程人员提供方便。

5.7 静态成员

在第 4 章中曾介绍过全局变量,使用全局变量可以起到数据共享的作用。但在面向对象的程序设计中,全局变量很少使用,因为全局变量的使用破坏了信息隐蔽性和封装性,不符合面向对象的思想,不利于程序的维护。在 **C++** 中常使用静态成员来实现对象间的数据共享。静态成员包含静态数据成员和静态成员函数。

在类的定义中,若在数据成员或成员函数前用关键字 **static** 修饰,这些成员就成为静态数据成员或静态成员函数。例如:

```
class myclass {
public:
    int fun1();           //非静态成员函数
    static float fun2(); //静态成员函数
private:
    int index;           //非静态数据成员
    static double value; //静态数据成员
};
```

5.7.1 静态数据成员

静态数据成员是同一个类中所有对象共享的数据成员,而不是某一个对象的数据成员。

在类中,在创建每一个对象时,对于一般的数据成员都分配独立的存储区域,以保存各自的值。而静态数据成员则不同,无论建立多少个该类的对象,一个类的静态数据成员只存储在唯一的一个存储单元中,从而实现同一类不同对象之间的数据共享。

静态数据成员的这种特征与 **C++** 程序在内存中的存储格局有关。一个 **C++** 程序在内存中的存储格局通常分为四个存储区。

1)全局数据存储区 也称为静态数据存储区。它存储程序中的全体常数、全局变量、静态变量(全局静态变量和局部静态变量),各类的静态数据成员也存放在该数据存储区内。存放在全局数据存储区中的数据具有生存周期长(自创建之时起直到程序执行结束时止)、缺省初

始值为 0 和存储区较大等特点。

2)栈式数据存储区 也称为局部数据存储区。它存储程序中各函数调用过程中的局部变量(包括函数形式参数和返回数据)。存放在栈式数据存储区中的数据具有生存周期短(自创建之时起直到所在程序块执行结束时止)、无缺省初始值、存储区较小等特点。

3)堆式数据存储区 也称为动态数据存储区。它存储程序中各函数执行过程中用 `new` 运算符或 `malloc()` 函数动态申请分配的数据。存放在堆式数据存储区中的数据具有生存周期不定(动态申请分配和动态释放)、无缺省初始值、存储区较大等特点。

4)代码存储区 存储程序中全体函数(包括成员函数和非成员函数)的可执行二进制代码。

从上述可知,由于一个类的静态数据成员唯一地存放在全局数据存储区中,所以同一个类中全体对象可以共享它。

另外,静态数据成员与运行的程序有相同的生存期,这一点很像全局变量。但是,它与全局变量不同。静态数据成员是类的成员,所以它遵从类的普通数据成员的访问规则。

静态数据成员必须在类体内说明,并且必须在任何类体和函数之外定义一次。定义时可以指定初始值,定义形式为:

类型 类名::静态数据成员; (缺省初始值为 0)

类型 类名::静态数据成员=初始值;

像普通数据成员一样,只有公有静态数据成员才可以被直接访问。访问的方法有以下几种:

类名::公有静态数据成员

对象名.公有静态数据成员

对象引用名.公有静态数据成员

对象指针->公有静态数据成员

需要注意,静态数据成员在类中只是进行了说明,并没有立即分配内存。因此,必须在程序的全局区域定义一次,以便为其分配内存,同时初始化。

例 5.20 静态数据成员的使用示例。

```
#include <iostream.h>
class M{
private:
    int A;
    static int B;
public:
    M(int a){A=a;B+=a;}    //构造函数,注意静态数据成员 B 值的变化
    void f1();              //成员函数说明
};
void M::f1()               //成员函数的定义
{
    cout<<"A="<<A;        //引用非静态数据成员
    cout<<"\tB="<<B<<endl; //引用静态数据成员
}
int M::B=0;
void main(){
    M P(5);                //把 P 的 A 初始化为 5,把静态数据成员 B 的值置为 5
    P.f1();                //输出结果:A=5    B=5
    M Q(10);               //把 Q 的 A 初始化为 10,把静态数据成员 B 的值置为 15
```

```

        Q.f1();           //输出结果:A=10    B=15
        P.f1();           //输出结果:A=5     B=15
    }

```

输出结果为:

```

A=5    B=5
A=10   B=15
A=5    B=15

```

若把主函数写成:

```

void main()
{
    M P(5);           //把 P 的 A 初始化为 5,把静态数据成员 B 的值置为 5
    M Q(10);          //把 Q 的 A 初始化为 10,把静态数据成员 B 的值置为 15
    P.f1();            //输出结果:A=5     B=15
    Q.f1();            //输出结果:A=10    B=15
}

```

输出结果为:

```

A=5    B=15
A=10   B=15

```

静态数据成员一般用于定义类的各对象所共用的数据,如计数、统计总数、平均数等。下例进一步表明静态数据成员的使用方法。

例 5.21 对象计数器:每创建一个对象,其值加 1;每释放一个对象,其值减 1。

```

#include<iostream.h>
class counter{
private:
    int objno;           //对象序号
public:
    static int count;     //对象计数器定义
    counter()            //定义构造函数
    {
        count++;         //每创建一对象,对象数加 1
        objno=count;     //设置当前对序号值
    }
    ~counter(){count--; }
    void show();
};

void counter::show()     //显示对象成员
{
    cout<<"obj"<<objno<<" "<<"count="<<count<<endl; }
int counter::count=0;    //静态数据成员初始化
void show(){ cout<<"当前对象计数器值为:"<<counter::count<<endl; }
void main()
{
    counter obj1;        obj1.show();
    counter *pc; pc=new counter;
    pc->show();obj1.show(); show();
    counter obj2,obj1.show(); obj2.show();pc->show();
    delete pc;show();
    counter obj3,obj3.show(); show();
}

```

```
}
```

运行结果为:

```
obj1 count=1
```

```
obj2 count=2
```

```
obj1 count=2
```

当前对象计数器值为:2

```
obj1 count=3
```

```
obj3 count=3
```

```
obj2 count=3
```

当前对象计数器值为:2

```
obj3 count=3
```

当前对象计数器值为:3

在上面的例子中,静态数据成员 `count` 被说明为公有的,这样,不仅可以为所创建的 `counter` 类的对象计数,而且在外函数 `show()` 中可以直接引用它们。

5.7.2 静态成员函数

在类定义中,前面带有 `static` 的成员函数称为静态成员函数,它同样也属于整个类,成为此类所有对象共享的成员函数,而不是属于类中的某个对象。它与非静态成员函数的区别是没有隐含的 `this` 指针参数。

如果是在类体内说明或定义静态成员函数,则必须在返回类型前面加上 `static`,而如果是在类体内说明、在类体外定义静态成员函数,则与一般成员函数定义相同,在定义函数时无需再加 `static`,也可以把它定义为内联函数。

静态成员函数可以直接访问所属类静态数据成员。由于它没有隐含的 `this` 指针参数,故不能直接访问非静态成员。一般静态成员函数主要用于访问全局变量或本类静态数据成员,特别是访问静态数据成员,起到在同一个类中对象之间的数据维护作用。通常情况下,静态成员函数不访问非静态成员。若确实需要访问非静态成员,只能通过对象参数访问它们。

下面举例说明静态数据成员和静态成员函数的意义。

例 5.22 某商店成箱地购进和卖出货物。货物以重量为单位,各箱的重量不同。编写程序记录库存货物的总重量。

将货物用一个类来描述。每购进一箱货,创建一个对象,记录其重量;而卖出一箱货物时,释放一个对象。货物的进出都要加减总重量和总箱数。

```
#include <iostream.h>
class Goods{
public:
    Goods(int w);
    ~Goods();
    static void PrintTotal();    //说明静态成员函数
private:
    int weight;
    static int TotalNumber;    //说明静态数据成员总箱数
    static int TotalWeight;    //说明静态数据成员总重量
};
Goods::Goods(int w)
{ weight=w;
  TotalWeight+=w;
```

```

    TotalNumber++;
};
Goods::~~Goods() { TotalWeight-=weight; TotalNumber--; }
void Goods::PrintTotal() //定义静态成员函数
{ cout<<"当前货物总箱数="<<TotalNumber<<"总重量="<<TotalWeight<<endl; }
int Goods::TotalWeight=0; //定义静态数据成员并初始化为 0
int Goods::TotalNumber=0; //定义静态数据成员并初始化为 0
Goods * Create() //创建一个 Goods 类对象并初始化
{ int t;
  cout<<"输入货物重量:"; cin>>t;
  Goods *p=new Goods(t); //调用构造函数
  return p;
}
void Erase(Goods *p){ delete p; } //调用析构函数释放一个 Goods 类对象
void main()
{ Goods *p1,*p2,*p3;
  p1=Create(); Goods::PrintTotal();
  p2=Create(); p2->PrintTotal();
  p3=Create(); p3->PrintTotal();
  Erase(p2); Goods::PrintTotal();
  Erase(p1); Goods::PrintTotal();
}

```

执行结果为:

```

输入货物重量:1000
当前货物总箱数=1,总重量=1000
输入货物重量:680
当前货物总箱数=2,总重量=1680
输入货物重量:1800
当前货物总箱数=3,总重量=3480
当前货物总箱数=2,总重量=2800
当前货物总箱数=1,总重量=1800

```

在本例中,每次使用 **new** 运算创建一个 **Goods** 类的对象,就调用一次该类的构造函数,使得表示货物总箱数和总重量的成员累加;而每次使用 **delete** 运算释放一个 **Goods** 类的对象,就调用一次该类的析构函数,使货物总箱数和总重量减少。为了查看当前货物总箱数和总重量,用“**Goods::PrintTotal()**”的形式调用静态成员函数,即用“类名::静态成员函数()”的形式调用,这是最常见的用法。也可以用“对象名.公有静态成员函数()”、“对象指针->静态成员函数()”等形式调用,如“**p3->PrintTotal()**”。

5.8 友元

在前面介绍类的数据成员时曾多次强调,只有类的成员函数才能直接访问类的私有和保护成员。当然,一个普通函数可以通过对象参数访问类中的公有成员,但这样容易破坏信息隐蔽的特性。

在某些情况下,可能需要在非类的成员函数中访问类的私有成员和保护成员。在 **C++** 中是用友元函数实现的。**C++** 中的友元也可以是一个类,即友元类。

一个类的普通成员函数的说明实际上描述了三个在逻辑上相互不同的性质:

- ①该函数能够访问类定义中的全体数据成员;
- ②该函数的作用范围仅仅局限于本类之内;
- ③该函数必须通过一个本类的对象调用(有一个 **this** 指针)。

说明为静态的成员函数只具有前两个性质,而说明为友元的函数只具有第一个性质。

使用友元破坏了类的封装性。引入友元的目的是加强、简化同类或异类对象之间的操作和提高面向对象程序的执行速度。

5.8.1 友元函数

友元函数要在一个类的类体内说明,形式为:

friend 类型名 友元函数名(形参表);

例如:

```
class Point{
private:
    double x,y;
public:
    Point(double, double);
    friend double Distance (Point &a,Point &b);
};
```

然后在类体外对友元函数进行定义,定义格式与普通函数相同,但可以通过对象名、对象指针和对象引用作为参数直接访问对象的私有成员。常常把对象引用作为形式参数。

例 5.23 用友元函数实现两个复数的加法运算。

```
#include <iostream.h>
class complex
{ private:
    double real,imag;
public:
    complex(double r=0,double i=0){real=r;imag=i;}
    void list();
    friend complex add(complex &x,complex &y);           //友元函数说明
};
void complex::list()
{ cout<<"("<<real;
  if(imag>=0)cout<<"+";
  cout<<imag<<"i)";
}
complex add(complex &x,complex &y)    //友元函数的实现
{ return complex(x.real+y.real,x.imag+y.imag); }
void main()
{ complex ob1(2,1.5), ob2(3.0,-6), obb;
  obb=add(ob1,ob2);
  ob1.list();cout<<"+";ob2.list(); cout<<"=";obb.list();cout<<endl;
}
```

输出结果为:

(2+1.5i)+(3-6i)=(5-4.5i)

在程序的类中,语句

```
friend complex add(comple&x x,complex &y);
```

的作用是说明 `add()` 函数是类 `complex` 的友元函数。该语句是函数 `add()` 的原型说明。该原型说明中 `friend` 后的 `complex` 表示 `add()` 函数返回的是 `complex` 类的对象。

上面的例子说明使用友元函数可以对多个同类对象进行操作。

下面的例子是用友元函数操作不同类的对象。

例 5.24 使用友元函数操作不同类的对象:计算平面上一个点 (x_0, y_0) 到已知一条直线 $Ax+By+C=0$ 的距离 $d=|Ax_0+By_0+C|/A^2+B^2$ 。

```
#include <iostream.h>
#include <math.h>
class Line;
class Point {
    double x,y;
    friend double PLDistance(Point &,Line &);    //友元函数说明
public:
    Point(double x1=0.0,double y1=0.0){ x=x1; y=y1; }
    void Print(){ cout<<"点("<<x<<","<<y<<") ";}
};
class Line {
    double A,B,C;
public:
    friend double PLDistance(Point &p,Line &l);    //友元函数说明
    Line(double a=0.0,double b=0.0,double c=0.0){ A=a; B=b; C=c; }
    void Print()
    {   cout<<"直线"<<A<<"x";
        if(B>=0.0)cout<<"+";   cout<<B<<"y";
        if(C>=0.0)cout<<"+";   cout<<C<<"=0";
    }
};
double PLDistance(Point &p,Line &l)    //友元函数的实现
{   return fabs(l.A*p.x+l.B*p.y+l.C)/sqrt(l.A*l.A+l.B*l.B);   }
void main()
{   Point p1(3.2,4.8);
    Line l1(2.0,3.1,-12.6);
    p1.Print(); cout<<"到"; l1.Print();
    cout<<"的距离="<<PLDistance(p1,l1)<<endl;
}
```

输出结果为:

点(3.2,4.8) 到直线 $2x+3.1y-12.6=0$ 的距离=2.35283

在上例中, 程序的第 3 行 `class Line` 称为前向声明, 表示类 `Line` 将在后面定义。

关于友元函数说明如下:

①必须在类体内说明友元函数,说明时以关键字 `friend` 开头,后跟友元函数的函数原型,友元函数的说明可以出现在类的任何地方,无论在 `private`、`protecte` 和 `public` 处, 意义都相同;

②注意友元函数不是类的成员函数,所以友元函数的实现与普通函数一样,在类体外实现时不用“::”指示属于哪个类,只有成员函数才使用“::”符号,如例 5.24 中的友元函数 `PLDistance()`;

③友元函数不能直接访问类的成员,只能访问对象的成员,如例 5.24 中 `x.real`、`temp.real`;

④友元函数可以访问对象的私有数据成员和保护数据成员,但普通函数不行;

⑤调用友元函数时,在实际参数中需指出要访问的对象,如例 5.24 中 `obb=add(ob1,ob2)` 语句中的 `ob1`、`ob2` 为具体对象;

⑥一个类的成员函数也可以作为其他类的友元函数,以便访问其他类的私有数据成员和保护数据成员。

5.8.2 友元类

不仅函数可以作为一个类的友元,而且一个类也可以作为另一个类的友元。这种友元类的说明方法是在另一个类的定义中加入如下形式的友元类说明语句:

`friend class` 友元类名;

该语句可以出现在另一个类的类体的任何部分。

当一个类被说明为另一个类的友元时,该类中的全部成员函数(包括构造函数和析构函数)均成为另一个类的友元函数。这就是说,一个友元类的所有成员函数都可以通过对象形参访问另一个类的全体数据成员。

例 5.25 友元类使用示例。

```
#include <iostream.h>
class X
{ friend class Y; //友元类说明
  int x;
  static int y;
public:
  void set(int i){x=i;y+=x;}
  void Display(){cout<<"x="<<x<<","y="<<y<<endl;}
};
class Y //友元类定义
{ int yy;
public:
  Y(int, X&); //友元类构造函数
  void Display(X);
};
int X::y=1; //定义静态数据成员并初始化
Y::Y(int i,X &a){ yy=i;a.x=a.x*i; }
void Y::Display(X a) { cout<<"x="<<a.x<<","y="<<X::y<<","yy="<<yy<<endl; }
void main()
{ X obx;
  obx.set(5);obx.Display();
  Y oby(8,obx);
  obx.Display();obx.set(9);
  oby.Display(obx);
}
```

输出结果为:

```
x=5,y=6
x=40,y=6
x=9,y=15,yy=8
```

从例中可以看出,友元类的成员函数中既可以通过对象参数访问另一个类的私有数据成员,也可以直接访问另一个类的私有或保护静态数据成员。

在设计程序时,应该根据需求选择成员函数、静态成员函数和友元函数。有些操作必须是成员函数,如构造函数、析构函数和后面要介绍的虚函数。由于成员函数的作用域是局限于类的,所以应用最多。而在设计运算符重载时,有时使用友元函数会更加有效。

相对而言,友元类不如友元函数使用得多。这是因为友元类使类之间的关系比较复杂,稍不注意就可能造成很难查找的错误。

5.9 类模板

第4章介绍过函数模板,这里介绍类模板(也称为类属)。模板(template)是C++支持参数化多态性的工具。所谓参数化多态性,是指将程序中所处理的对象的类型参数化,从而使得编写的一个模板(函数或类)可以适应处理多种不同类型对象的需要。也就是说,使用模板可以更容易地编写高效、可靠、高适应性的通用函数和通用类。因此,它是使用C++实现软件重用的一种重要机制。由于这种参数化类型的出现,产生了“范型”程序设计(或称为“类属”程序设计),给计算机软件的发展注入了新的活力。

一个类模板允许用户为类定义一种模式,使得类中的某些数据成员、某些成员函数的参数、某些成员函数内部所需要的局部变量以及某些成员函数的返回值能取任意类型(包括系统预定义的类型和用户自定义的类型)。

本节首先介绍类模板的说明和类模板的实例化。C++标准中所包含的STL(Standard Template Library 标准模板库)将在数据结构部分介绍。

5.9.1 类模板说明

类模板说明的形式如下:

```
template<模板参数表> class 类名{ ... };
```

其中,模板参数表中的项的两种基本形式为:

```
class 或 typename 标识符 (指明可以接受一个类型参数)
```

```
基本类型名 标识符 (指明可以接受一个由指定类型的常量作为参数)
```

对于第一种形式的模板参数, **class** 和 **typename** 是等价的,在它们后面出现的标识符(即类型参数名)必须至少在对应的类模板定义中出现一次。这种形式的模板参数所对应的实例化参数是一个任意类型名。对于第二种形式的模板参数,它们后面出现的标识符在对应的类模板定义中作为常量使用。这种形式的模板参数所对应的实例化参数是一个指定类型的常数。

当模板参数表同时包含上述多项内容时,各项内容要用逗号隔开。应该注意,如果类模板中类的成员函数在类体外实现时,它们必须是模板函数,即该函数定义的前面必须带有“**template<模板参数表>**”,同时,在模板类名与作用域运算符之间写上“**<模板参数表>**”。

类模板说明(以及函数模板说明)本身并不产生代码,它只是指定了一个类族。只有它被引用时,才根据实例化时数据类型的需要产生相应代码。

例 5.26 一个简单的数组类模板说明。

```
#include<iostream.h>
//数组类模板说明
template <class T,int n>
```

```

class Array{
public:
    Array(){ for(int k=0;k<n;k++) Buffer[k]=0; }    //构造函数
    void SetElem(int i,T v);                        //设置第 i 个数组元素值为 v
    T   GetElem(int i);                             //取出第 i 个数组元素值
private:
    T   Buffer[n];                                  //长度为 n 的 T 型数组
};
template <class T,int n>                            //类 Array 的成员函数
void Array<T,n>::SetElem(int i,T v){
    if(i>=0&& i<n) Buffer[i]=v;                    //进行下标的合理性检查
    else cout<<"设置数组元素值时下标("<i<<"太大或太小。\\n";
}
template <class T,int n>                            //类 Array 的成员函数
T Array<T,n>::GetElem(int i){
    char c=0;
    if (i>=0 && i<n) return Buffer[i];             //进行下标的合理性检查
    else
    { cout<<"取出数组元素值时下标("<i<<"太大或太小, 设为 0。\\n";
      return c;
    }
}
}

```

在上面这个例子中, 模板参数 **T** 表示一个可变化的类型名字。**Array** 是一个类名。在 **Array** 类的声明过程中, 类型模板参数 **T** 可以作为数据成员、成员函数的函数参数、成员函数内局部变量和成员函数返回值的类型, 如 “**T Buffer[n]**、**T v**、**T GetElem(int i)**”等。模板参数 **i** 表示一个整型常量, 它作为一维 **T** 型数组 **Buffer** 的长度使用。但是, 它的值对于每个对象又可以不同。

从上面这个例子可以看出, 类模板说明的形式与类的定义形式很相似。主要的差别在于, 类模板说明必须用“**template<模板参数表>**”开始, 并且在其后的类声明中引用模板参数, 从而说明了一种类中数据成员、函数参数、函数返回值等的类型可以“任意”变化的参数化类。

5.9.2 类模板的实例化

将类模板实例化为一个具体的类并建立对象的形式是:

类名<实际类型表> 对象名 1,...,对象名 n;

其中, 类名是在类模板说明中指定的类名, 通常把实例化的类称为模板类。在实际类型表中要列出实例化后的类名、类型名或常数, 且各实际类型或常数要与类模板说明中的“模板参数表”中的参数一一对应。编译程序会按照指定的实际类型生成一个实在的类, 并创建该类的对象。

例 5.27 数组类模板的实例化示例。

```

#include<iostream.h>
//数组类模板说明
template <class T,int n>
class Array{
public:
    Array(){ for(int k=0;k<n;k++) Buffer[k]=0; }    //构造函数

```

```

void SetElem(int i,T v);           //设置第 i 个数组元素值为 v
T GetElem(int i);                 //取出第 i 个数组元素值
private:
    T Buffer[n];                  //长度为 n 的 T 型数组
};
template <class T,int n>           //类的成员函数
void Array<T,n>::SetElem(int i,T v){
    if(i>=0&& i<n) Buffer[i]=v;   //进行下标的合理性检查
    else cout<<"设置数组元素值时下标("<i<<"太大或太小。\\n";
}
template <class T,int n>           //类的成员函数
T Array<T,n>::GetElem(int i){
    char c=0;
    if (i>=0 && i<n) return Buffer[i]; //进行下标的合理性检查
    else
        { cout<<"取出数组元素值时下标("<i<<"太大或太小，设为 0。\\n";
          return c;
        }
}
void main(){
    int i;
    Array<int,3> m;                //实例化为整型数组类
    for (i=0;i<3;i++){
        m.SetElem(i,i*10);
        cout<<m.GetElem(i)<<" ";
    }
    cout<<endl;
    m.SetElem(5,-1);
    Array<double,5> x;             //实例化为双精度型数组类
    for(i=0;i<5;i++){
        x.SetElem(i,i+10.5);
        cout<<x.GetElem(i)<<" ";
    }
    cout<<endl;
    x.GetElem(i);
    char p[]="abcd";
    Array< char,4 >s;              //实例化为字符型数组类
    for(i=0;i<4;i++){
        s.SetElem(i,p[i]);
        cout<<s.GetElem(i)<<" ";
    }
    cout<<endl;
}

```

输出结果为:

0 10 20

设置数组元素值时下标(5)太大或太小。

10.5 11.5 12.5 13.5 14.5

取出数组元素值时下标(5)太大或太小，设为0。

a b c d

本例用 `Array<int,3>` 创建了一个具有 3 个数组元素的整型数组对象 `m`，用 `Array<double,5>` 创建了一个具有 5 个数组元素的双精度型数组对象 `x`，用 `Array<char,4>` 创建了一个具有 4 个数组元素的字符型数组对象 `s`。

可以看出，类模板给用户提供了一种强大工具，不仅能够说明适应于多种类型的通用类，而且能够避免 C 语言中某些低级成分的缺陷。如果对程序的执行速度要求不太严格，就应该对数组进行下标合理性检查，使数组更加安全可靠。在缺省情况下，为了保证最快的执行速度，C++是不对数组进行下标合理性检查的。

5.10 例题

例 5.28 写出下面程序的输出结果。

```
#include <iostream.h>
class M
{ int x,y;                //私有数据成员
public:
    M(){x=y=0;}           //缺省构造函数
    M(int i, int j){x=i;y=j;}
    void setxy(int i, int j){x=i;y=j;}
    void copy(M *m);
    void print(){cout<<"x="<<x<<"y="<<y<<endl;}
};
void M::copy(M *m) { x=m->x;y=m->y;} //指向对象的指针作函数的形参
void fun(M m1,M *m2,M &m3)         //对象、对象指针、对象引用作函数的形参
{ m1.setxy(10,20);
  m2->setxy(11,22);
  m3.setxy(1,2);
}
void main()
{ M mp(3,9),mq(33,44),mr;
  cout<<"mp:";mp.print();
  cout<<"mq:";mq.print();
  mr.copy(&mp);
  cout<<"mr:";mr.print();
  fun(mp,&mq,mr);
  cout<<"mp:";mp.print();
  cout<<"mq:";mq.print();
  cout<<"mr:";mr.print();
}
```

输出结果为:

```

mp:x=3,y=9
mq:x=33,y=44
mr:x=3,y=9
mp:x=3,y=9
mq:x=11,y=22
mr:x=1,y=2

```

例 5.28 写出下面程序的输出结果。

```

#include <iostream.h>
class B{
    int a, b;
public:
    B(int i, int j){a = i; b = j; cout << "B 构造 ";}
    ~B(){cout << "B 析构 "; print();}
    void print(){cout << "(" << a << ", " << b << ")\\n";}
};
class A{
    int x;
    B s;
public:
    A(int i, int j, int k):s(i, j),x(k){ cout << "A 构造 \\n";}
    ~A(){cout << "A 析构 \\n";print();}
    void print(){cout << "x=" << x << ", "; s.print();}
};
void main() { A p(4, 2, 6); }

```

输出结果为:

```

B 构造 A 构造
A 析构
x=6 ,(4,2)
B 析构 (4,2)

```

例 5.31 使用友元函数计算一个点到一个圆的圆心的距离,并利用它判断一个点在圆外、圆内还是圆上。

```

#include<iostream.h>
#include<math.h>
class Circle;
class Point{
    double x,y;
public:
    Point (double x1,double y1){ x=x1; y=y1;}
    void print() {cout<<"("<<x<<","<<y <<")";}
    friend double Distance (Point a,Circle b); //友元函数说明
    friend int inCircle (Point a,Circle b);
};
class Circle{
    double x,y,r;

```

```

public:
    Circle(double x1,double y1,double r1){ x=x1; y=y1; r=r1;}
    friend double Distance (Point a,Circle b);
    friend int inCircle (Point a,Circle b);
    void print(){cout<<"["<<x<<" "<<y <<" "<<r<<"]";}
};
//友元函数 Distance 定义
double Distance(Point a,Circle b){ //通过对象参数引用对象的私有成员
    double dx=a.x-b.x;
    double dy=a.y-b.y;
    return sqrt(dx*dx+dy*dy);
}
int inCircle (Point a,Circle b){
    if(Distance(a,b)<b.r) { cout<<"在圆";b.print(); cout<<"内\n"; return -1; }//在圆内
    if(Distance(a,b)==b.r) { cout<<"在圆";b.print(); cout<<"上\n"; return 0; }//在圆
    if(Distance(a,b)>b.r) { cout<<"在圆";b.print(); cout<<"外\n"; return 1; } //在圆外
}
void main()
{ Point p1(3,4),p2(5,4);Circle c(6,8,5);
  cout<<"点"; p1.print();cout<<"到圆";c.print();
  cout<<"的距离=" <<Distance(p1,c)<<endl;
  cout<<"点"; p1.print(); inCircle (p1,c);
  cout<<"点"; p2.print();cout<<"到圆";c.print();
  cout<<"的距离=" <<Distance(p2,c)<<endl;
  cout<<"点"; p2.print(); inCircle (p2,c);
}

```

上

运行结果为:

点(3,4)到圆[(6,8),5]的距离=5

点(3,4)在圆[(6,8),5]上

点(5,4)到圆[(6,8),5]的距离=4.12311

点(5,4)在圆[(6,8),5]内

练习 5

1.回答以下问题:

- (1)什么是类,什么是对象?
- (2)面向对象程序设计方法的基本特征是什么?
- (3)类与结构有什么相同点和不同点?
- (4)类中用 **private** 说明的成员和用 **public** 说明的成员有什么不同?
- (5)构造函数和析构函数各有什么特性和作用?什么是拷贝构造函数,何时调用它?
- (6)什么是子对象?如果一个类含有子对象,如何对子对象进行初始化?
- (7)**this** 指针有什么作用? ***this** 代表什么含义?
- (8)静态成员分为哪两种,各有什么特点?
- (9)友元函数有什么特性,友元函数的作用是什么?
- (10)什么是类模板,类模板的作用是什么?它与普通的类定义有什么不同?

2.单项选择题。

- (1)以下叙述中不正确的是_____。
 - A) 类中的数据成员可以是私有或公有的,而类中的成员函数必须是公有的
 - B) 拷贝构造函数的作用是使用一个已经存在的对象去初始化一个新的同类的对象
 - C) 类中的构造函数可以重载,而析构函数不能重载
 - D) 构造函数和析构函数都应是类的公有成员函数
- (2)以下关于类和对象叙述正确的是_____。
 - A) 一般只有通过具体的对象才能访问类的成员函数
 - B) 类和对象间没有联系
 - C) 对象是抽象的,而类是具体实现
 - D) 一个类的成员函数可以任意被调用
- (3)_____不是构造函数的特征。
 - A) 构造函数的函数名与类名相同
 - B) 构造函数可以重载
 - C) 构造函数可以设置缺省参数
 - D) 构造函数必须指定类型说明
- (4)下列函数中,_____不能重载。
 - A) 成员函数
 - B) 非成员函数
 - C) 析构函数
 - D) 构造函数
- (5)下列函数中,_____不是类的成员函数。
 - A) 友元函数
 - B) 拷贝构造函数
 - C) 析构函数
 - D) 构造函数
- (6)类 A 的成员函数为 `void Set(A &a)`,其中 `A &a` 的含义为_____。
 - A) `a` 是类 A 的对象引用,用作函数 `Set()` 的形参
 - B) 将 `a` 的函数地址赋给函数 `Set`
 - C) 变量 A 与 `a` 按位相“与”作为函数 `Set()` 的形参
 - D) 指向类 A 的指针为 `a`

3.写出以下程序的输出结果。

```
(1)#include <iostream.h>
class AA
{ public:
    AA(int i,int j)
    {   A=i; B=j;
        cout<<"Constructing("<<A<<","<<B<<").\n";
    }
    ~AA() { cout<<"Destructed("<<A<<","<<B<<").\n"; }
    void print(){ cout<<A<<","<<B<<endl; }
private:
    int A,B;
```

```
};
void main()
{ AA *a1,*a2;
  a1=new AA(1,2);
  a2=new AA(5,6);
  a1->print(); a2->print();
  delete a1; delete a2;
}
```

(2)#include<iostream.h>

```
class A{
    static int count;
public:
    A(){ ++count; }
    ~A(){ --count; }
    static int f(){ return count; }
};
```

```
int A::count=0;
void main()
{ cout<<A::f()<<" ";
  A a;
  A *p=new A;
  cout<<A::f()<<" ";
  delete p;
  cout<<A::f();
}
```

4.程序填空。

(1)下面的程序输出小于 200 的素数，而且每行最多输出 10 个素数。

```
#include<iostream.h>
#include<iomanip.h>
class Prime {
    int p;
public:
    Prime(int n){if(n<3)p=3; else p=n; }
    void Run();
};
void Prime::Run()
{ int k,j,flag,line=0;
  for(j=2;_____;j++)
  { flag=1;
    for(k=2;k<j;k++)
      if(j%k==0)flag=_____;
    if(flag)
    { cout<<setw(5)<<j;
      line++;
    }
```

//line 为已输出素数个数

```

        if( _____ )cout<<endl;
    }
}
cout<<endl;
}
void main(){ Prime obj( _____ ); _____; }

```

(2)以下程序能实现求 $a^2+b^2+c^2+\dots$ 。其中 a 、 b 、 c的值由对象的初始化值提供。该程序使用静态数据成员实现。

```

#include <iostream.h>
class myclass {
public:
    myclass(int x);
    void getnumber();
    void getresult();
private:
    int a;
    static long int sum;           //定义静态数据成员
};
myclass::myclass(int x) { a=x; sum+=a*a; }           //构造函数
void myclass::getnumber(){ cout<<"Number="<<a<<endl;}
void myclass::getresult(){ cout<<"Result="<<sum<<endl; }
_____ ;
void main()
{ myclass ob1(5),ob2(7),ob3(9);
  ob1.getnumber(); ob2.getnumber(); ob3.getnumber();
  _____getresult();
}

```

5.扩充日期类 **Date** 的定义,增加一个成员函数 **NumberOfDays()**,计算当前对象中的日期是其年份的第几天,并编写一个主函数测试这个成员函数。

6.定义一个名为 **Integer** 的整数类,具有数据成员 d 、成员函数 **GetD()**获取 d 的值、**SetD(int)**设置 d 的值、**IsOdd()**判断 d 是否为一个偶数、**IsPrime()**判断 d 是否为一个素数,并设计主函数用一个对象分别设置 d 的值为 15 和 31,编写一个主函数测试这个类。

7.编写一个求 $n!$ 的类,并在 **main()**函数中分别输出 2~9 的阶乘。

8.设计一个平面直线类 **line**,采用友元函数判断两条直线是平行还是相交,并采用友元函数在相交时的交点坐标。再编写一个主函数进行测试。